Отчет
Индивидуального Домашнего Задания №3
Вариант 35

Сделано
Жарковой Татьяной Владимировной
студенткой 2 курса Программной инженерии
группы БПИ-253

Москва 2026
Высшая школа экономики

Оглавление

Аннотация 4

Формулировка варианта № 35 4

Формулировка задания № 3 5

Решение задания 10

Общие изменения 10

`model.h` 10

`StopReason` 11

`Team` 11

`Call` 12

`Simulation` 12

От sync.h к ipc.h 13

От `sync.h` к `message.h` 13

`MsgType` 14

`Role` 14

`BoxKind` 15

`Settings` 16

`Counters` 17

`sems.h` 18

`roles.h` 20

`signals.h` 20

utils_extra.h и utils.c 21

`config.c` 22

`dispatch.c` 22

`generator.c` 22

`simulation.c` 23

`roles.c` 23

signals.c 24

`main_server.c` 24

`main_client.c` 25

Версии 26

Аннотация

В данном отчете представлена работа индивидуального домашнего задания, вариант 35. Работа выполнена на языке C в личном репозитории.

Программа представляет собой событийную симуляцию работы диспетчерской службы. Она моделирует поступление экстренных вызовов, поиск подходящей свободной бригады, выезд бригады к месту происшествия, выполнение работ, завершение обслуживания и возвращение бригады на базу.

Формулировка варианта № 35

В диспетчерскую службу поступают вызовы из разных районов города. Каждый вызов имеет уникальный номер, координаты, категорию срочности и требуемую специализацию бригады. Бригады различаются специализацией, скоростью движения и текущим местоположением. Диспетчер назначает свободную подходящую бригаду. После назначения бригада едет к месту вызова, оказывает помощь и затем становится доступной в текущем районе либо возвращается на базу. Срочность вызова может повышаться по мере ожидания. Один вызов не должен быть одновременно назначен нескольким бригадам, если условием явно не требуется совместная работа.

Требуется разработать консольное приложение, моделирующее работу службы экстренной помощи.

Входные параметры

• карта районов и время перемещения;
• количество и специализации бригад;
• число, время и место поступления вызовов;
• категории срочности;
• длительность оказания помощи;
• правила повышения приоритета;
• стратегия назначения и возврата бригад;

• максимальное время ожидания.

Отображаемые события

Необходимо отображать поступление вызова, изменение срочности, поиск и назначение бригады, выезд, прибытие, начало и окончание помощи, освобождение бригады, отмену и вызов, оставшийся без обслуживания.

Инварианты

• одна бригада обслуживает не более одного вызова;
• назначенная бригада соответствует требуемой специализации;
• один обычный вызов назначается не более чем одной бригаде;
• завершенный или отмененный вызов не назначается повторно;
• состояние доступности бригады изменяется согласованно с её маршрутом и работой.

Завершение

Моделирование завершается после окончания поступления вызовов и обработки всех возможных заявок либо по завершении смены с фиксацией необслуженных вызовов, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения.
Диспетчер, каждая бригада и каждый вызов являются самостоятельными участниками модели.

Формулировка задания № 3

Минимальные требования к выполнению задания

Третье задание заключается в разработке программы, состоящей из взаимодействующих процессов. При этом используются знания и навыки, полученные в ходе изучения основ программирования, языка программирования C и разделов данной дисциплины. Необходимо обратить внимание на формирование консольного интерфейса программы,

ее общей структуры, используя по возможности решения полученнные в ходе выполнения предыдущих заданий.

Так как большинство задач связаны с моделированием различных ситуаций, необходимо обеспечить наглядное представление результатов. Для интерактивного и наглядного отображения на дисплее рекомендуется использовать временные задержки.
В целом принятые решения определяется условием задачи и фантазией разработчика.

В ходе выполнения задания необходимо обеспечить следующие требования, позволяющие получить минимальную положительную оценку.

1. Предоставить отчет, в котором отразить следующую информацию:
• привести фамилию, имя, отчество исполнителя, группу;
• указать номер варианта и условие задачи;
• представить сценарий, поясняющий, каким образом исходные сущности и их поведение отображаются в потоки и их взаимодействие;
• описать реализацию пользовательского интерфейса;
• описать варианты запуска программы и ее завершения в зависимости от используемых исходных данных.

2. Программа должна корректно выполняться в соответствии с условием задания.

3. Программа должна быть реализована на языке программирования C с использованием библиотек из libc, системных вызовов.

4. Вывод программы должен быть информативным, отражая все ключевые протекающие в ней события в терминах моделируемой предметной области. Наблюдатель на основе вывода программы должен понимать, что в ней происходит в каждый момент времени ее работы.

5. Независимо от способа запуска программа должна завершаться в соответствии с возможными вариантами данных.

6. По возможности процессами должны быть представлены все моделируемые сущности, обладающие поведением или их семейства. Если по мнению разработчика такое решение невозможно, то число процессов и их ролевое распределение осуществляется по согласованию с преподавателем.

7. Процессы должны асинхронно взаимодействовать между собой, что необходимо поддерживать использованием изученных средств взаимодействия. Их использование определяет минимальные требования к организации программы.

8. Результаты работы программы должны быть отражены в отчете.

Функциональные требования на оценку в 4-5 баллов

Разработать консольное приложение, в котором единый родительский процесс запускает требуемое число дочерних процессов.

Требования:
1. Представить сценарий решаемой задачи поясняющий, каким образом исходные сущности и их поведение отображаются в процессы и их взаимодействие.

2. В зависимости от выбранной схемы решения программа может использовать обмен данными через общую или разделяемую память. Применение семафоров в программе возможно по одному из следующих вариантов:

• Множество процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через разделяемую память в стандарте POSIX.

• Множество процессов взаимодействуют с использованием неименованных POSIX семафоров расположенных в разделяемой памяти. Обмен данными также ведется через разделяемую память в стандарте POSIX.

3. Реализовать завершение программы в соответствии с условием задачи, а также предусмотреть корректное завершение по прерыванию с клавиатуры по соответствующему сигналу.

4. В программе предусмотреть удаление семафоров и разделяемой памяти по ее завершению любым из способов.

Функциональные требования на оценку в 6 баллов

В дополнение к программе, разработанную на предыдущую оценку, необходимо разработать программу, в которой для взаимодействия процессов используется оставшийся вариант из двух, описанных выше, но не реализованный на предшествующую оценку.

Функциональные требования на оценку в 7-8 баллов

В дополнение к программам на предыдущие оценки необходимо разработать еще одну программу, состоящую из отдельных программ–процессов, запускаемых независимо друг от друга. С использованием семафоров и разделяемой памяти организовать между ними взаимодействие.

Требования:
1. Для завершения приложения использовать те же подходы, что и в предыдущих решениях.

2. Множество независимых процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через разделяемую память в стандарте POSIX.

3. В отчет расширить информацией о добавленной реализации и привести соответствующие результаты работы программы.

Функциональные требования на оценку выше 8 баллов

В дополнение к программам на предыдущие оценки необходимо разработать приложение, состоящую из отдельных программ–процессов,

запускаемых независимо друг от друга. С использованием семафоров и каналов (или очередей сообщений) организовать между ними взаимодействие.

Требования:

1. Для завершения приложения использовать те же подходы, что и в предыдущих решениях.

2. Множество независимых процессов взаимодействуют с использованием именованных POSIX семафоров. Обмен данными ведется через каналы или очереди сообщений в стандарте POSIX.

3. Отчет расширить информацией о добавленной реализации и привести соответствующие результаты работы программы.

Дополнительные требования

К дополнительным требованиям, влияющим на повышение оценки, относятся следующие.

1. В программе присутствуют комментарии, поясняющие выполняемые действия и описание используемых объектов и переменных.

2. Реализация ввода–вывода с применением системных вызовов и функций библиотеки libc, использующих дескрипторы файлов.

3. Независимо от способа запуска программа должна завершаться с использованием как по завершению работы, так и по ее прерыванию. Использование сигналов и изменение с их помощью условий прерывания программы повышает оценку.

4. Сохранение лога по завершении работы, к котором отражаются результаты выполнения, например, собранная статистика.

5. Формирование тестовых наборов данных и скриптов, обеспечивающие их запуск.

6. Предоставление при использовании ИИ всех промтов, приведших к формированию программного кода.

7. Критическая оценка программы, реализованной с использованием ИИ. Фиксация в отчете правок кода, если таковые были.

8. Реализация непредусмотренных в этом списке решений, которые могут впечатлить преподавателя и сподвигнуть его на повышение оценки.

Решение задания

Общие изменения

Переход от потоков к процессам потребовал заменить все средства синхронизации, работавшие только внутри одного адресного пространства, на межпроцессные. Мьютексы `pthread_mutex_t` заменены двоичными семафорами, условные переменные больше не используются, а почтовые ящики, которые раньше хранились внутри структур `Team` и `Call`, перенесены в отдельный IPC-слой. Этот слой предоставляет единый интерфейс (`ipc.h`), за которым скрываются четыре реализации: `fork` с именованными семафорами, `fork` с неименованными семафорами, независимые процессы с разделяемой памятью и независимые процессы с очередями сообщений. Логика работы сущностей (часы, диспетчер, бригада, вызов) осталась прежней, но теперь она выполняется в отдельных процессах, а не в потоках. Для обмена сообщениями введены новые типы, которые используются только в версии с очередями сообщений, где нет общей памяти.

`model.h`

`StopReason`

```c
/**
 * @enum StopReason
 * @brief Why the simulation loop has stopped
 */
typedef enum
{
    STOP_ALL_PROCESSED = 0, // the event queue became empty
    STOP_SHIFT_LIMIT,       // the next event is after the end of the shift
    STOP_INTERRUPTED,       // SIGINT / SIGTERM from the user
    STOP_INTAKE_CLOSED,     // SIGQUIT: no new calls, active calls were finished
    STOP_PROCESS_FAILURE    // a process of the simulation died unexpectedly
} StopReason;
```

Добавляется неожиданное завершение дочернего процесса. Мы должны обнаружить такую ситуацию и корректно завершить симуляцию.

`Team`

```c
/**
 * @struct Team
 * @brief Team in the simulation
 */
typedef struct
{
    int id;
    Specialization spec;
    int speed;
    int district;
    int base;
    TeamState state;
    int call_id;

    // process of the team (its mailbox / message queue lives in the IPC layer, see ipc.h)
    pid_t pid;   // pid of the process that plays this team (0 = not started yet)
    int started; // 1 if the process is running and registered

} Team;
```

Идентификатор потока заменён идентификатором процесса, удалено поле `Mailbox` (механизм обмена сообщениями теперь находится в отдельном IPC-слое).

`Call`

```c
/**
 * @struct Call
 * @brief Call in the simulation
 */
typedef struct
{
    int id;
    int district;
    int x, y; // coordinates of the call location
    int spec;
    int priority;
    int arrival;
    int service_duration;
    int max_wait;
    int escalation_after;
    int cancel_after;
    CallState state;
    int assigned_team;
    int dispatch_time;
    int service_start;
    int completion_time;

    // process of the call: own process (fork versions) or the "callers" family process
    pid_t pid;   // pid of the process that plays this call (0 = not started)
    int started; // 1 if the process that plays the call is running

} Call;
```

Идентификатор потока заменён идентификатором процесса, удалено поле `Mailbox` (механизм обмена сообщениями теперь находится в отдельном IPC-слое).

`Simulation`

```c
    // processes
    pid_t clock_pid;        // pid of the clock process, clients forward signals to it
    pid_t dispatcher_pid;   // pid of the dispatcher process
    pid_t callers_pid;      // pid of the callers process
    int processes_started;  // number of started/registered processes
    char log_path[LINE_LEN];// log file, opened by every process with O_APPEND
```

Удален файловый дескриптор (каждый процесс самостоятельно открывает лог с флагом `O_APPEND`), полностью удален блок многопоточной синхронизации. Вместо него появился блок процессов:

- `clock_pid` - PID главного процесса, который управляет модельным временем и очередью событий;
- `dispatcher_pid` - PID процесса диспетчера;
- `callers_pid` - PID процесса семейства callers;

- `processes_started`- количество запущенных и зарегистрированных процессов;
- `log_path` - путь к лог-файлу.

То есть сам принцип симуляции практически не меняется. Меняется способ исполнения. Вместо потоков происходят отдельные процессы и их взаимодействия.

От sync.h к ipc.h

Раньше `sync.h` самостоятельно определял структуры, содержащие `pthread_mutex_t`,` pthread_cond_t` и `sem_t`.

Теперь `ipc.h` объявляет функции, через которые остальная программа взаимодействует с IPC. Это что-то вроде абстрактного интерфейса.

Существуют 2 способа взаимодействия:
1. Через разделяемую память - процессы видят общие данные;
2. Через очереди сообщений - у каждого процесса своя копия модели, которую нужно синхронизировать сообщениями.

Вместо хранения почтового ящика внутри каждой сущности программа обращается к IPC-слою, который сам определяет, куда и как доставлять сообщения.

От `sync.h` к `message.h`

Новый файл. Раньше описание сообщений находилось в `sync.h`, но теперь оно было вынесено в отдельный `message.h`. Там находятся не только типы сообщений, но и роли процессов, адреса получателей и форматы передаваемых данных. Описывается, что передается: типы сообщений, роли процессов, адреса получателей, формат сообщения.

`MsgType`

```c
/**
 * @enum MsgType
 * @brief Kinds of messages that the processes send to each other
 */
typedef enum
{
    MSG_WAKE = 0,         // clock -> call/team: an event of this entity has come (Message.event)
    MSG_NEW_CALL,         // call -> dispatcher: the call has arrived and waits (arg = call index)
    MSG_CALL_CHANGED,     // call -> dispatcher: the priority of a waiting call has changed
    MSG_TEAM_FREE,        // team -> dispatcher: the team is available again (arg = team index)
    MSG_ROUND,            // clock -> dispatcher: all reactions of this minute are done, assign teams
    MSG_ASSIGN,           // dispatcher -> team: serve the call (arg = call index)
    MSG_CALL_FINISHED,    // team -> call: the help is finished (arg = call index, value = team index)
    MSG_SHUTDOWN,         // clock -> everybody: the simulation is over (not counted as work)

    // the messages below are used only by the message-queue version (no shared memory)
    MSG_SYNC,             // new copy of a call and/or team record for the mirror of the receiver
    MSG_SCHEDULE,         // any -> clock: put event (event, arg = id, value = time) into the queue
    MSG_REMOVE_DEADLINES, // any -> clock: remove the deadlines of call arg from the queue
    MSG_DONE,             // any -> clock: one counted message has been handled completely
    MSG_HELLO,            // client -> clock: a new process has registered (sender_role / sender_idx)
    MSG_WELCOME,          // clock -> client: configuration path and command-line overrides
    MSG_STATS             // client -> clock: final counters of the process before it exits
} MsgType;
```

Появились новые типы сообщений, которые используются только в версии с очередями, где нет общей памяти:

- `MSG_SYNC` - новая копия записи вызова или бригады для получателя;
- `MSG_SCHEDULE` - просьба к часам поставить событие в очередь;
- `MSG_REMOVE_DEADLINES` - просьба к часам удалить сроки вызова из очереди событий;
- `MSG_DONE` - обработано одно сообщение»;
- `MSG_HELLO` - клиент зарегистрировался, сообщает свою роль и PID;
- `MSG_WELCOME` - часы передают клиенту путь к конфигурации и параметры командной строки;
- `MSG_STATS` - клиент перед выходом отдаёт часам свои счетчики статистики.

`Role`

```
/**
 * @enum Role
 * @brief Roles of the processes
 */
typedef enum
{
    ROLE_CLOCK = 0,  // the server: clock, event queue, invariants, signals, summary
    ROLE_DISPATCHER, // assigns free teams to waiting calls
    ROLE_TEAM,       // one team (index = team index)
    ROLE_CALL,       // one call (fork versions, index = call index)
    ROLE_CALLERS,    // the family of all calls (independent versions)
    ROLE_COUNT
} Role;
```

Новое перечисление Role содержит:

- `ROLE_CLOCK`;
- `ROLE_DISPATCHER`;
- `ROLE_TEAM`;
- `ROLE_CALL`;
- `ROLE_CALLERS`;
- `ROLE_COUNT`.

Роль процесса определяет, какой почтовый ящик он читает и как обрабатывает сообщения.

`BoxKind`

```
/**
 * @enum BoxKind
 * @brief Address of a mailbox
 */
typedef enum
{
    BOX_DISPATCHER = 0,
    BOX_TEAM,  // + team index
    BOX_CALL,  // + call index
    BOX_CLOCK  // only in the message-queue version
} BoxKind;
```

Перечисление `BoxKind` задаёт адрес получателя:

- `BOX_DISPATCHER`;
- `BOX_TEAM`;
- `BOX_CALL`;

- `BOX_CLOCK`.

Раньше отправитель передавал указатель на почтовый ящик, теперь передаётся «кому» (вид + индекс), а IPC-слой сам определяет ящик или очередь.

`Settings`

```c
/**
 * @struct Settings
 * @brief Command-line overrides sent to the clients of the message-queue version
 */
typedef struct
{
    int delay_ms;
    int shift_limit;
    int strategy;
    int random_calls;
    unsigned int seed;
    pid_t clock_pid;
    char config_path[LINE_LEN];
    char log_path[LINE_LEN];
} Settings;
```

Структура Settings содержит параметры командной строки, которые часы передают клиентам в версии с очередями сообщений:

- `delay_ms`;
- `shift_limit`;
- `strategy`;
- `random_calls`;
- `seed`;
- `clock_pid`;
- `config_path`;
- `log_path`.

Клиент читает тот же файл конфигурации и применяет те же параметры, поэтому его копия модели совпадает с моделью часов.

`Counters`

```c
/**
 * @struct Counters
 * @brief Statistics collected by one process
 */
typedef struct
{
    long total_wait;
    long total_service;
    int served;
    int cancelled;
    int unserved;
    int escalations;
    int dispatches;
    int received;
} Counters;
```

Структура `Counters` содержит статистику одного процесса:

- `total_wait`;
- `total_service`;
- `served`;
- `cancelled`;
- `unserved`;
- `escalations`;
- `dispatches`;
- `received`.

В версии с очередями у каждого процесса свои счётчики, и в конце часы суммируют их.

```c
/**
 * @struct Message
 * @brief One message of a mailbox (shared memory) or of a message queue
 */
typedef struct
{
    int type;        // MsgType
    int event;       // EventType for MSG_WAKE / MSG_SCHEDULE
    int arg;         // index of a call or a team (id for MSG_SCHEDULE / MSG_REMOVE_DEADLINES)
    int value;       // extra value: team index for MSG_CALL_FINISHED, time for MSG_SCHEDULE
    int time;        // model time of the sender when the message was sent
    int sender_role; // Role of the sender
    int sender_idx;  // index of the sender (team index), -1 if not applicable

    int call_idx;
    int team_idx;
    int call_count;
    Call call;
    Team team;

    union
    {
        Settings settings; // MSG_WELCOME
        Counters counters; // MSG_STATS
    } u;
} Message;
```

Структура `Message` была из трёх полей, стала структурой фиксированного размера без указателей.
Поле type теперь int, а не `MsgType`. Поле `value` - дополнительное значение (индекс бригады для `MSG_CALL_FINISHED`, время для `MSG_SCHEDULE`). Поле `time` - модельное время отправителя. В версии с очередями у каждого процесса своё `sim->now`, и получатель берет текущее время из сообщения. Поля `sender_role`, `sender_idx` - кто отправил. Поля `call_idx`, `team_idx`, `call_count`, `call`, `team` - свежие копии записей для получателя. `Union u` - данные для `MSG_WELCOME` и `MSG_STATS`.

`sems.h`

Новый файл. Таблица всех семафоров программы. У каждого семафора есть номер `semId`, остальная программа получает семафор только через `sem_ref(id)` и не знает, именованный он или нет.

```c
/**
 * @enum SemId
 * @brief Numbers of the semaphores
 */
typedef enum
{
    SEM_WORLD = 0,        // binary: lock of the shared model
    SEM_OUTPUT,           // binary: lock of the console and the log file
    SEM_QUIET_LOCK,       // binary: lock of the counter of unhandled messages
    SEM_QUIET_IDLE,       // posted when the counter of unhandled messages drops to 0
    SEM_READY,            // counting: client processes that finished registration
    SEM_EXITED,           // counting: client processes that finished their work
    SEM_SENT,             // counting: messages sent so far
    SEM_CLAIM_DISPATCHER, // 1 = the role of the dispatcher is free
    SEM_CLAIM_CALLERS,    // 1 = the role of the callers family is free
    SEM_CLAIM_TEAM0,      // + team index: 1 = the team is not played by a process yet
    SEM_BOX0 = SEM_CLAIM_TEAM0 + MAX_TEAMS, // 3 per mailbox: lock, filled, free
    SEM_TOTAL = SEM_BOX0 + 3 * BOX_COUNT
} SemId;
```

Перечисление `SemId` содержит:

- `SEM_WORLD`;
- `SEM_OUTPUT`;
- `SEM_QUIET_LOCK`;
- `SEM_QUIET_IDLE`;
- `SEM_READY`;
- `SEM_EXITED`;
- `SEM_SENT`;
- `SEM_CLAIM_DISPATCHER`;
- `SEM_CLAIM_CALLERS`;
- `SEM_CLAIM_TEAM0 + k`;
- `SEM_BOX0 + 3 * BOX_COUNT`.

Мьютексы `pthread_mutex_t` заменены двоичными семафорами (начальное значение 1, `sem_wait` = захват, `sem_post` = освобождение). Мьютекс по умолчанию работает только внутри одного процесса, а задание требует синхронизации семафорами. Объявлены функции работы с таблицей семафоров: создание, получение, проверка существования, удаление. Две реализации (`sem_named.c` и `sem_unnamed.c`) описаны в разделах соответствующих оценок. Общие для обеих реализаций `sem_wait_retry` и `sem_wait_ms` лежат в `sem_common.c`.

`roles.h`

```c
#ifndef ROLES_H
#define ROLES_H

#include "message.h"
#include "model.h"

// Main loop of a dispatcher, team, call or callers process
int role_loop(Simulation *sim, Role role, int idx);

#endif
```

Новый файл, заменяет `thread_internal.h`. Объявлена функция `role_loop` - главный цикл процесса диспетчера, бригады, вызова или callers (до `MSG_SHUTDOWN`). Было четыре функции потоков и структура аргументов `ThreadArgs`: `call_thread`, `team_thread`, `dispatcher_thread`, `signal_thread`. `ThreadArgs` больше не нужна: поток получал аргументы через указатель на память, выделенную `malloc`, а процесс получает их просто как параметры `role_loop` — после `fork` у ребёнка есть копия всех переменных родителя. Четыре функции потоков заменены одним циклом `role_loop(sim, role, idx)`, который по роли выбирает обработчик. Функция потока сигналов ушла в `signals.h`.

`signals.h`

```c
/**
 * @struct ControlRequests
 * @brief Requests of the user received with signals by the clock process
 */
typedef struct
{
    int stop;        // SIGINT / SIGTERM: stop now
    int stop_signal; // which of them came first
    int close;       // SIGQUIT: close the intake
    int snapshot;    // SIGUSR1: print the live status (reset after reading)
} ControlRequests;
```

Новый файл. Структура `ControlRequests` заменяет четыре флага, которые раньше лежали в `Simulation` (`stop_requested`, `close_requested`, `snapshot_requested`). Добавлено поле `stop_signal`: какой сигнал пришёл первым, чтобы часы написали в лог правильный текст.

Объявлены функции:

- `signals_install_server` - часы устанавливают обработчики `SIGINT`, `SIGTERM`, `SIGQUIT`, `SIGUSR1` с `sigaction()`;
- `signals_take` - читает запросы, запрос снимка сбрасывается;
- `signals_stop_requested` - 1, если получен `SIGINT` или `SIGTERM` (для циклов ожидания);
- signals_install_child` - для дочерних процессов версий с `fork` (`Ctrl+C` и `Ctrl+\` игнорируются, `SIGTERM` и `SIGUSR1` пересылаются часам);
- `signals_install_client` - для независимых клиентов (`SIGINT`, `SIGTERM`, `SIGQUIT`, `SIGUSR1` пересылаются часам);
- `signal_text` - имя сигнала для лога.

utils_extra.h и utils.c

```c
// Sets the identification tag of the current process for log messages
void set_process_tag(const char *tag);

// Descriptor of the log file of the current process
void set_log_fd(int fd);
int get_log_fd(void);

// Locks the output semaphore to prevent mixed output of several processes
void output_begin(Simulation *sim);

// Unlocks the output semaphore after the output operation is completed
void output_end(Simulation *sim);
```

Функция `set_thread_tag` переименована в `set_process_tag`: метка в логе теперь обозначает процесс (роль). Добавлены `set_log_fd` и `get_log_fd`: дескриптор лога хранится не в модели, а в переменной процесса. В комментариях к `output_begin` и `output_end` мьютекс заменён на семафор.

В `utils.c` `set_log_fd` - новая функция, запоминает дескриптор лог-файла текущего процесса; вызывается в `main_server.c` и `main_client.c` после `open()`. `get_log_fd` возвращает дескриптор лога текущего процесса; используется в `print_summary` и в `main_server.c` при закрытии файла. `output_begin`: было - захват мьютекса из модели, стало - захват семафора `output` через IPC-слой.

`config.c`

Изменена только функция `set_defaults`. Удалена строка `sim->log_fd = -1`, потому что поля `log_fd` в модели больше нет. Значение «лога нет» теперь задаётся начальным значением `log_descriptor = -1` в `utils.c`.

`dispatch.c`

Изменена только функция `plan_dispatch`. Изменилась одна строка: `remove_call_deadlines` заменена на `ipc_remove_deadlines`. После назначения у вызова больше не должно быть сроков ожидания (эскалация, отмена, тайм-аут). В версиях с разделяемой памятью `ipc_remove_deadlines` сразу удаляет их из общей очереди событий, а в версии с очередями сообщений диспетчер не видит очередь событий и отправляет часам просьбу `MSG_REMOVE_DEADLINES`.

`generator.c`

Изменена только функция `find_free_call_slot`.Функция ищет слот для нового случайного вызова. Если массив вызовов заполнен, используется слот закрытого вызова. Раньше его поток останавливался и присоединялся (`join_call_thread`), теперь процесс останавливается через `ipc_stop_call`: в версиях с `fork` - сообщение `MSG_SHUTDOWN` и `waitpid` процесса вызова; в независимых версиях - у вызова нет своего процесса, слот просто помечается свободным.

`simulation.c`

Потока сигналов больше нет, флаги выставляют обработчики сигналов в самом процессе часов, поэтому функцию заменили на `signals_take()` из `signals.c`. `spawn_random_call`: генератор создаёт запись случайного вызова и запускает его исполнителя. Раньше создавался поток (`start_call_thread`), теперь `ipc_start_call`: в версии с `fork` - `fork` нового процесса вызова; версия на 7–8 - вызов помечается как обслуживаемый процессом callers (он видит запись в разделяемой памяти); версия > 8 —-дополнительно отправляется копия записи процессам `callers` и диспетчеру (`MSG_SYNC`), потому что общей памяти нет. Если запустить исполнителя не удалось, вызов записывается как необслуженный (текст ошибки изменён: «no process could be created»). `deliver`: доставляет событие владельцу - `MSG_WAKE` бригаде (прибытие, конец помощи, возвращение) или вызову (поступление, эскалация, отмена, тайм-аут).

`run_simulation`: главный цикл часов. Алгоритм тот же: прочитать запросы пользователя, взять события ближайшей минуты, разбудить владельцев, дождаться тишины, раунд диспетчера, дождаться тишины, проверить инварианты. `set_thread_tag` заменён на `set_process_tag`; все `pthread_mutex_lock/unlock` заменены на `world_lock/unlock`; в строке `SHIFT STARTED` вместо `threads` теперь `processes`; запросы пользователя читаются через `signals_take` вместо `read_control`.

`roles.c`

Три файла с функциями потоков объединены в один `roles.c`. Реакции сущностей перенесены почти без изменений: строки, которые меняют состояние вызовов и бригад и пишут в лог, совпадают построчно. Изменилось то, что вокруг реакции: поток сам крутил бесконечный цикл `for (;;) { mailbox_get(..); ... }`, теперь цикл один на все роли — `role_loop`, а реакции вынесены в функции `call_handle`, `team_handle`, `dispatcher_handle`, которые обрабатывают одно сообщение. Мьютексы

`pthread_mutex_lock/unlock` заменены на `world_lock/unlock`. `schedule_event_checked` заменён на `ipc_schedule`, `remove_call_deadlines` - на `ipc_remove_deadlines`. `send_message` заменён на `ipc_send_simple. `quiet_done` переехал из каждой функции потока в `role_loop` (`ipc_done`). После изменения записи вызывается `ipc_publish_call` / `ipc_publish_team`. В версиях с разделяемой памятью эти функции ничего не делают (запись и так общая), а в версии с очередями рассылают новую копию записи другим процессам.

signals.c

- `signal_thread` заменена обработчиками сигналов.;
- `server_handler` - обработчик в процессе часов: выставляет флаги, ничего не печатает;
- `forward_handler` - обработчик в остальных процессах: пересылает полученный сигнал часам через `kill`;
- `set_handler` - вспомогательная функция установки обработчика через `sigaction()`;
- signals_install_server` - устанавливает обработчики часов на четыре сигнала и игнорирует `SIGPIPE`;
- `signals_take` - копирует флаги в структуру `ControlRequests`, сбрасывает флаг снимка;
- `signals_stop_requested` - возвращает 1, если получен SIGINT/SIGTERM;
- `signals_install_child` - для дочерних процессов версий с `fork`: `SIGINT` и `SIGQUIT` игнорируются, `SIGTERM` и `SIGUSR1` пересылаются родителю;
- `signals_install_client` - для независимых клиентов: все четыре сигнала пересылаются часам;
- `signal_text`- возвращает текст для строки в логе.

`main_server.c`

Один `main()` используется всеми четырьмя программами-часами.
Различие в реализации IPC-слоя, с которой он собран.
Порядок работы остался прежним:

1. умолчания;
2. аргументы;
3. конфигурация;
4. аргументы ещё раз;
5. проверка;
6. лог;
7. запуск исполнителей;
8. симуляция;
9. остановка;
10. итоги;
11. освобождение ресурсов.

`main_server.c` - это точка входа процесса часов. Один и тот же `main()` используется всеми четырьмя программами-часами: `ed_fork_named`, `ed_fork_unnamed`, `ed_shm_clock` и `ed_mq_clock`. Различие между ними заключается только в том, с какой реализацией IPC-слоя файл собран: с `ipc_shm.c` и именованными семафорами, с `ipc_shm.c` и неименованными семафорами, с `ipc_shm.c` для независимых процессов или с `ipc_mq.c` для очередей сообщений.

`main_client.c`

Новый файл. Из одного исходника собираются три программы, роль задаётся при компиляции через `-DCLIENT_ROLE=ROLE_DISPATCHER`, `ROLE_TEAM` или `ROLE_CALLERS`. Тот же файл используется версией выше 8 баллов, где он собирается с `ipc_mq.c`. Клиентские программы - это `ed_shm_dispatcher`, `ed_shm_team`, `ed_shm_callers` и соответствующие им `ed_mq_*`.

`main_client.c`: это независимый процесс, который не создаёт IPC-объекты и не управляет модельным временем. Он находит уже существующий сеанс по имени, захватывает свободную роль, регистрируется в модели и

дальше обрабатывает сообщения своей роли. В версии с разделяемой памятью клиент после подключения работает с общей моделью. В версии с очередями сообщений у клиента есть своя копия модели, которую он загружает по тому же файлу конфигурации и параметрам из `MSG_WELCOME`, а изменения получает и рассылает сообщениями `MSG_SYNC`. Клиент не удаляет IPC-объекты: это делает только сервер.

Версии

Версия на 4–5 баллов (`ed_fork_named`) использует `fork` и именованные POSIX семафоры, обмен данными идет через разделяемую память POSIX. Родительский процесс (часы) создаёт объект POSIX shm, кладёт в него модель и почтовые ящики, создает именованные семафоры и порождает через `fork` процесс диспетчера, процесс на каждую бригаду и процесс на каждый вызов (в том числе для вызовов случайного генератора — прямо во время симуляции).

Версия на 6 баллов (`ed_fork_unnamed`) отличается только тем, что вместо именованных семафоров используются неименованные POSIX семафоры, расположенные в разделяемой памяти .

Версия на 7–8 баллов (`ed_shm_*`) состоит из независимых программ, которые пользователь запускает в разных терминалах. Часы создают объекты и ждут регистрации клиентов; клиенты находят объекты по имени сеанса, захватывают роль и регистрируются. Используются именованные семафоры и разделяемая память POSIX.

Версия выше 8 баллов (`ed_mq_*`) также состоит из независимых программ, но разделяемой памяти нет совсем: все данные передаются через очереди сообщений POSIX (`mq_open`, `mq_send`, `mq_receive`), по одной очереди на роль. Каждый процесс хранит свою копию модели и синхронизирует её сообщениями `MSG_SYNC`. Очередь событий есть только у часов, остальные процессы просят часы поставить или удалить событие (`MSG_SCHEDULE`, `MSG_REMOVE_DEADLINES`).

Статистика собирается в конце: каждый процесс отправляет свои счётчики часам (`MSG_STATS`), часы их складывают. Для синхронизации используются именованные семафоры (`output`, `sent`, `ready`, `exited`, `claim_*`). Счётчик тишины построен на семафоре `sent` и сообщениях `MSG_DONE`: фаза закончена, когда число обработанных сообщений равно числу отправленных.